

Paradigmas de programación Informe N°2

Alvaro Benjamín Reyes Encalada

Profesor:
Víctor Flores

Santiago - Chile
2021

Tabla de Contenidos

CAPÍTULO 1. Introduccion	3
CAPÍTULO 2. Marco teorico	4
2.1 Paradigma	4
2.1.1 Solución del Problema.....	4
2.1.2 El paradigma lógico.....	5
2.2 El problema.....	6
CAPÍTULO 3. Desarrollo de la aplicación	7
3.1 Generalidades previas	7
3.2 TDA	7
3.3 Predicados	8
3.3.1 Socialnetworkregister	8
3.3.2 Socaialnetworklogin	8
3.3.3 Socialnetworkpost	8
3.3.4 Socialnetworkfollow	9
3.3.5 Socialnetworkshare.....	9
3.3.6 socialnetworkToString.....	9
3.3.7 Comment	9
3.3.8 Socialnetworklike	9
CAPÍTULO 4. Análisis de resultados	10
CAPÍTULO 5. Instrucciones de uso.....	11
CAPÍTULO 6. Conclusiones.....	12
CAPÍTULO 7. Referencias.....	13

Tabla de Imágenes

Imagen 1. Creación de Socialnetwork.....	8
Imagen 2. Socialnetworkpost	8
Imagen 3. SocialnetworkToString.....	9
Imagen 4. Cargar archivo	11

CAPÍTULO 1. INTRODUCCION

A los alumnos de ingeniería en informática de la Universidad de Santiago de Chile, dentro del transcurso del curso llamado Paradigmas de la Programación, se les ha propuesto dar solución a un mismo problema, pero visto de distintos puntos de vista, es decir, cambiar la forma en la cual se le da solución al problema, pero más que ser formas de solucionar problemas, un paradigma es una forma de pensar, por lo que la dificultad de este curso, no es tanto solucionar el problema, sino, cambiar la forma en la cual hay que pensar para dar solución al mismo.

Dentro del siguiente informe se procederá a explicar cómo se ha dado una solución el problema expuesto, pero con la diferencia, de que esta solución ahora se ha propuesto desde un modelo de pensamiento distinto, un paradigma lógico, y como su nombre lo indica, se basa en los conceptos de lógica matemática, predicados que caracterizan o relacionan a los individuos involucrados y la deducción de las posibles respuestas a una determinada consulta ya que, en sí, es un paradigma declarativo.

CAPÍTULO 2. MARCO TEORICO

2.1 PARADIGMA

Un concepto importante de programación lógica es la descomposición de programas en sus componentes lógicos y sus componentes de control. Con lenguajes de programación lógica de bajo nivel, estos componentes determinan la solución del problema, por eso los componentes de control pueden variar para proporcionar alternancia de ejecución de un programa lógico. Estos conceptos son capturados con el eslogan

Algoritmo= lógica + control

donde "lógica" representa un programa lógico y "control" diferentes estrategias de demostración del teorema.

2.1.1 Solución del Problema

Es un razonamiento que determina un árbol lógico, el cual constituye un espacio de búsqueda para resolver el problema. El nivel superior es la raíz del árbol. Dado algún nodo en el árbol y alguna cláusula cuya cabecera está marcada por un nodo, existe un conjunto de nuevas ramas correspondientes con los objetivos del cuerpo de las cláusulas, aún por reducir. Estas nuevas ramas son agrupadas juntas por el "y" lógico. El conjunto alternativo correspondiente de estas nuevas ramas está agrupado por el "o" lógico.

Alguna estrategia de búsqueda puede ser usada para la búsqueda del conjunto de proposiciones a resolver. Prolog es un lenguaje de programación que usa una secuencia, último en entrar primero en salir, para una estrategia de agrietamiento, en la cual la alternativa y el objetivo son considerados a la vez. Otras estrategias de búsqueda son por ejemplo la búsqueda paralela o la búsqueda primero-mejor para encontrar una solución óptima.

En el caso más general, donde los objetivos comparten variables, otras estrategias pueden ser usadas, como por ejemplo puede ser escogido el objetivo más altamente instanciado o el suficientemente instanciado para lo que implica un procedimiento. Una estrategia actualmente usada es la programación de lógica concurrente.

2.1.2 El paradigma lógico

La programación lógica permite formalizar hechos del mundo real, por ejemplo:

- ✓ las aves vuelan
- ✓ los pingüinos no vuelan
- ✓ "pichurri" es un ave
- ✓ "sandokan" es un perro
- ✓ "alegría" es un ave

y también reglas o restricciones:

- ✓ una mascota vuela si es un ave y no es un pingüino

Ante dicho "programa" es posible establecer hipótesis que no son más que preguntas o incógnitas, por ejemplo:

- ✓ ¿"pichurri" vuela?
- ✓ ¿qué mascotas vuelan?

Gracias a que la lógica de primer orden es computable, el ordenador será capaz de verificar la hipótesis, es decir, responder a las incógnitas:

- ✓ Es cierto que "pichurri" vuela.
- ✓ "pichurri" y "alegría" vuelan.

Obsérvese que el programa lógico no solamente es capaz de responder si una determinada hipótesis es verdadera o falsa. También es capaz de determinar que valores de la incógnita hacen cierta la hipótesis.

Este ejemplo es claramente académico. Sin embargo, consideremos el siguiente ejemplo: el sistema de control de semáforos de una ciudad.

El estado de cada uno de los semáforos (verde, rojo o ámbar) constituye los hechos del mundo real. El programa en sí consiste en unas pocas reglas de sentido común: determinados semáforos no pueden permanecer simultáneamente en verde, un semáforo solamente puede transitar de verde a ámbar y de ámbar a rojo, etc. La hipótesis es el estado en el que deberían estar cada uno de los semáforos en el siguiente instante de tiempo.

Este es un ejemplo imposible de resolver mediante programación tradicional, ya que la lógica subyacente al comportamiento de los semáforos en su conjunto queda enmascarada por simples órdenes imperativas del tipo "cambiar color de tal o cual semáforo".

2.2 EL PROBLEMA

El proyecto para presentar se basa en realizar una red social, en donde, se muestra cómo realizar ciertas operaciones que pueden realizar esta plataforma. Las redes sociales son estructuras compuestas por un grupo de actores (personas, empresas, fundaciones, bots, etc.) interconectados a través de relaciones uni- o bi- direccionales. (Laboratorio, 2021 – Paradigmas). Entonces, cómo realizar esta red social en base a las operaciones a realizar (ver capítulo 3) será problemática que tenemos para poder prestar una solución asociada a una red social.

CAPÍTULO 3. DESARROLLO DE LA APLICACIÓN

Dentro del siguiente capítulo se le otorgará una visión general del programa, como fue programado y que problemas resuelven sus partes entre las consultas aplicadas para el Laboratorio.

3.1 GENERALIDADES PREVIAS

Para programar la aplicación se ha utilizado el lenguaje de programación denominado Prolog, el cual es un lenguaje lógico, lo que quiere decir que es posible incluir más de un paradigma dentro del lenguaje. Además, para la creación de vistas y main se incluirá todo bajo concepto único de un solo archivo con extensión “.PL”, llamado “*Lab2Prolog_183921517_ReyesEncalada.pl*”. Este main cuenta con vistas que se encargan de interactuar con los predicados implementados en los TDA que van a consultar.

3.2 TDA

Para la implementación de los predicados, se posiciona cada uno bajo las siguientes entradas:

1. socialnetworkregister
2. socialnetworklogin
3. socialnetworkpost
4. socialnetworkfollow
5. socialnetworkshare
6. socialnetworkToSting

Predicados extras para laboratorio:

1. comment
2. Socialnetworklike

3.3 PREDICADOS

Hablamos de predicados a los hechos realizados para este laboratorio, el cual se desarrolla dos hechos previos llamados “Fecha” y “SocialNetwork”, los cuales llaman al formato de la fecha y a la creación como tal de la red social llamada Failbok.

```
% fecha(D,M,Y,F):- Fecha en el formato D/M/Y volviendo a F.
fecha(D,M,Y,F):- F = date(D,M,Y).

% Creacion de la red social (socialNetwork).
socialNetwork(Name,Date,Sn2):- Sn2 = [Name,Date,[],0].
```

Imagen 1. Creación de Socialnetwork

3.3.1 Socialnetworkregister

Se registra un usuario la red Social Network, mediante la obtención de 1 o más usuarios a registrar, para posterior realizar las funciones correspondientes a una red social como lo es postear, seguir, compartir, dar «me gusta» (like), etc.

3.3.2 Socaialnetworklogin

Login a través de la red social, donde un usuario “X” inicia sesión a través de failbok.

3.3.3 Socialnetworkpost

Publicar cualquier estado, es decir, texto, imagen, video, archivo, etc.... a la red social. Cada validación de este predicado obtiene información proporcionada desde los selectores desarrollados, como, por ejemplo: “getpost” y “getpostbypostID” y bajo ese contexto, en los modificadores se ha desarrollado “updatepost” que ayuda a enviar una publicación a los nombres de usuario que se quiera o a la lista de contactos que este lo estén siguiendo.

```
Constructor
socialNetworkPost(Sn1,Date,Post,List,Sn2):- Sn1 = [Name,_,Users,_],updatePost(Sn1,Post,List,Users,Final),Sn2 = [Name,Date,Final,0].

Selectores
getPost([],_):- [].
getPost(Users,PostId,Post):- Users = [H|_],H = [_,_PostId,_],Post1 = [Post,PostId,_].
getPost(Users,PostId,Post):- Users = [H|T],getPost(T,PostId,Post),H = [_,_PostId,_],not(Post1 = [Post,PostId,_]).
getPostbypostId(Sn1,Post,PostId):- getUsers(Sn1,Users),getPost(Users,PostId,Post).

Modificadores
updatePost(Sn1,Text,Follow,Users,Final):- loggedin(Sn1,User),Users = [H|_],H = [H1,Pass,[Exist,[],0,[0],Follow],User = H1,append([Text],Exist,New).
updatePost(Sn1,Text,Follow,Users,Final):- loggedin(Sn1,User),Users = [H|T],H = [H1,_,_],not(User = H1),updatePost(Sn1,Text,Follow,T,Final).
```

Imagen 2. Socialnetworkpost

3.3.4 Socialnetworkfollow

Sigue un determinado nombre de usuario en la red social, la cual esta se va actualizando cada vez que se añade un seguidor gracias al predicado “updatefollowers” desarrollada en los modificadores, donde se selecciona, añade y agrega un nuevo usuario a seguir.

3.3.5 Socialnetworkshare

Predicado que realiza el compartir de un mensaje asociado a un ID del usuario, bajo el predicado “updateshare” que ayuda a compartir una publicación a los usuarios deseados.

3.3.6 socialnetworkToString

Convertir los datos en cadena para una mejor comprensión y más conveniente, la cual permite obtener una representación del TDA “socialNetwork” como un string posible de visualizar de forma comprensible al usuario.

```

Constructor:
socialnetworkToString(Snl,String):- get_name(Snl,Name),get_date(Snl,date(0,M,Y)),getUsers(Snl,Users),length(Users,TotalUsers), get_usernames(Users,Usernames), string_join(Usernames,',',UsernamesStr)
    string_join(["Nombre de la red social: ",Name,"\\n","Registro ",date(0,M,Y),"\\n","Total de usuarios registrados: ",TotalUsers,"\\n"],StringA), string_join([StringA,"\\n","Nombre de usuario: ",UsernamesStr],String).

Modificador:
    string_join([],Return,Final):- Final = Return.
string_join(List,X,Final):- List = [_:H|_:T], string_concat(X,' ',X1),string_concat(X1,H,X2),string_join(T,X2,Final).

```

Imagen 3. SocialnetworkToString

3.3.7 Comment

Básicamente agrega un comentario al post, donde el predicado “updatecomment” ayuda a añadir y actualizar los comentarios de un posteo en particular.

3.3.8 Socialnetworklike

Like por una publicación o un comentario en la red social (social network), donde “updatelike” hace que te guste un post o comentario.

CAPÍTULO 4. ANÁLISIS DE RESULTADOS

Una vez terminado el proyecto, la funcionalidad cumple con los requisitos funcionales establecidos, a continuación se procederá a evaluar cada uno de ellos con lo que la aplicación posee:

1. Una implementación de TDA como constructor del laboratorio.
2. Se han implementado predicados en base a los TDA.
3. Creación de consultas, para utilizar todos los predicados de “consulta” en los hechos realizados.
4. Verificación entre selectores y modificadores en apoyo a los TDA en base a predicados realizados y los hechos a consultar.
5. Se implemente predicados Comment y socialnetworklike para desbordamiento de puntaje.
6. Los selectores utilizados, proporciona en detalle de todos los usuarios registrados, quienes se han conectado recientemente a través de una fecha determinada de cuando hace login un usuario y realiza algún posteo o comentario en un post en particular. Estos selectores están adecuados básicamente como “get” para proporcionar información.
7. Los modificadores fueron utilizados para actualizar la información proporcionada por cada acción o hecho a realizar como, por ejemplo, seguir un usuario nuevo o dar like a un post y aumentar su cantidad de likes.
8. “string_join”, ayuda como predicado a socialnetworkToString para crear una cadena a partir de una lista en base a los predicados construidos y retorna a nueva variable para actualizar.
9. Se ocupa un solo archivo .pl para la construcción del laboratorio.
10. Cada cambio realizado en SWI-Prolog, se aplicaba cambio en GITHUB.

CAPÍTULO 5. INSTRUCCIONES DE USO

Se solicita construir una red social en un lenguaje de programación llamado Prolog, almacenando operaciones que típicamente se pueden realizar en estas plataformas, las cuales se encuentran:

1. Crear una cuenta
2. Iniciar sesión
3. Añadir/Seguir amigos/persona/empresa/página
4. Eliminar/Dejar de seguir amigos/persona/empresa/página
5. Crear/Eliminar/Editar una publicación
6. Responder una publicación
7. Compartir una publicación
8. Enviar mensaje privado
9. Reaccionar a una publicación (like)

El programa presentado permite encontrar cada predicado desde un hecho a consultar que indica propiamente tal la salida, las cuales las publicaciones, usuarios registrados, seguidores o reacciones dentro de una publicación.

El programa puede ser utilizado tanto en Linux como en Windows, con el mismo IDE.

Swi-prolog es el IDE para utilizar en su versión 8.2.3, la cual tenemos que ir a *file – consult* y cargamos nuestro archivo .pl para realizar las consultas correspondientes. (ver imagen 4)

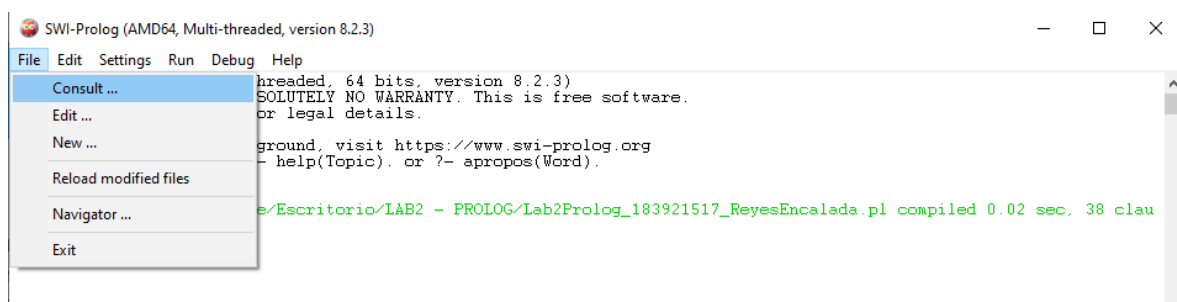


Imagen 4. Cargar archivo

Luego, podemos realizar las consultas, las cuales están en el archivo “LAB2 – Consultas.txt”.

Si queremos ver el código desde la suite como tal, debemos ir a *file – edit* y aparecerá el código propiamente tal.

CAPÍTULO 6. CONCLUSIONES

Para finalizar, nuevamente la mayor dificultad dentro del desarrollo de la aplicación fue la planificación del proyecto en base al nuevo paradigma, el cambio en la manera de pensar el desarrollo de una aplicación es tremendamente complicado, pero una vez que se ha podido establecer el nuevo paradigma, el resto de las dificultades no es tan grande (dificultades como: aprender un lenguaje de programación nuevo, definición entre funciones).

Como se menciona en la sección de análisis de resultados, se ha podido entregar una aplicación, la cual funciona satisfactoriamente implementado en el paradigma lógico, luego de terminar esta experiencia, se resaltan las facilidades otorgadas por el paradigma, en realidad, se resalta la lógica en definir para la utilización de nuevos hechos en base a los predicados, ya que estos hacen mucho más accesible la resolución de problemas.

Concluyendo, el paradigma lógico, es una buena herramienta para crear soluciones a problemas, tanto por su planificación como por sus predicados para diseñar un sin fin de plataformas bajo un hecho, cómo consultas por una red social al registrarse, seguir, compartir, like, hacer un post o comentario en particular.

CAPÍTULO 7. REFERENCIAS

Páginas web:

https://es.wikipedia.org/wiki/Programaci%C3%B3n_l%C3%B3gica (Wikipedia, 2020, pág. 3)

<https://wiki.uqbar.org/wiki/articles/paradigma-logico.html> (UQBAR, 2013, pág. 2)

<https://swish.swi-prolog.org/p/Tutorial%20de%20prolog.swinb>

<https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&ved=2ahUKEwjNwcrIwNnxAhVuqZUCHavIA8QQFjADegQICBAD&url=http%3A%2F%2Fferestrepoca.github.io%2Fparadigmas-de-programacion%2Fproglogica%2Ftutoriales%2Fprolog-gh-pages%2FTutorialProlog2017I.pdf&usg=AOvVaw10XUwuTw81VpofUpjkMdWv> (tutorial prolog)

https://www.researchgate.net/publication/263273690_Reglas_Difusas_en_Prolog

<https://blog.adrianistan.eu/supertutorial-prolog>

Material moodle USACH – Paradigmas de programación 2021.